Use of AI assistants. Problems 3.1 to 3.4 are No AI. Problem 3.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Racket file**, built from the template: code as code, traces and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper.

Problem 3.1 (Standard higher-order functions). No AI Write the three functions below with the standard higher-order functions and **no explicit recursion**. `equal?` is given in the template.

(a) A loan is a name and a number of days overdue. `late-fees` lists the overdue borrowers, each with a fee of 5 per day.

```
(late-fees '(("Ann" 0) ("Bo" 3) ("Cy" 12))) ; '(("Bo" 15) ("Cy" 60))
(late-fees '(("Ann" 0)))                    ; '()
```

(b) An order is affordable when every price is positive and the total is within the budget.

```
(affordable? '(120 350) 500) ; #t
(affordable? '(120 -5) 500)  ; #f
(affordable? '(300 350) 500) ; #f
```

(c) `run-lengths` replaces each run of equal neighbours by the element and the length of the run. This undoes the `expand` of the lecture.

```
(run-lengths '(a a b c c c)) ; '((a 2) (b 1) (c 3))
(run-lengths '(a b a))       ; '((a 1) (b 1) (a 1))
(run-lengths '())            ; '()
```

Problem 3.2 (Higher-order functions of your own). No AI Consider a bank that applies a rule to a balance once a year, a shop that pairs quantities with unit prices, and a list to be cut into stretches of alike elements. Write, **by explicit recursion**, three functions that take a function as an argument. `same-length?` is given in the template.

(a) `forecast`: a balance after a number of years, when the same rule is applied to it once a year. This is the lecture's `twice`, with the number of times as a parameter.

```
(forecast (lambda (b) (* b 11/10)) 3 1000) ; 1331
(forecast (lambda (b) (- b 100)) 2 1000)   ; 800
(forecast (lambda (b) (* b 2)) 0 7)        ; 7
```

(b) `combine`: two lists paired off with a given function, until one runs out.

```
(combine * '(2 3) '(350 120))                        ; '(700 360)
(combine string-append '("Ann" "Bo") '("a" "b" "c")) ; '("Anna" "Bob")
```

(c) `group`: the list cut into stretches of consecutive elements that a given test says are alike, which is `run-lengths` with the comparison as a parameter and the elements kept.

```
(group = '(1 1 2 3 3 3))                          ; '((1 1) (2) (3 3 3))
(group same-length? '("tea" "cat" "milk" "rice")) ; '(("tea" "cat") ("milk" "rice"))
(group = '())                                     ; '()
```

Problem 3.3 (A proof for every list). No AI Consider `my-map` and two functions:

```
(define (my-map f lst)
  (cond [(empty? lst) empty]
        [else (cons (f (first lst)) (my-map f (rest lst)))]))

(define (inc x) (+ x 1))
(define (dbl x) (* 2 x))
```

(a) Check by two traces that these give the same list:

```
(my-map dbl (my-map inc '(1 2)))
(my-map (lambda (x) (dbl (inc x))) '(1 2))
```

(b) Prove that, for *every* pair of functions `f` and `g` and every list `lst`,

```
(my-map f (my-map g lst)) = (my-map (lambda (x) (f (g x))) lst)
```

in the style of the lecture. A list is `empty` or `(cons z zs)`, so there are two cases, and in the second you may assume the claim for the shorter list `zs`. One step of the proof applies a `lambda` to an argument. Write that step out.

(c) In one sentence: which side does less work on a list of a thousand elements, and why?

Problem 3.4 (A pipeline, read and rearranged). No AI Consider a list of orders, each a name, a quantity and a unit price, and a pipeline over it:

```
(define orders '(("tea" 2 350) ("soap" 1 120) ("rice" 3 90)))

(define (revenue orders)
  (foldl + 0
         (map (lambda (o) (* (second o) (third o)))
              (filter (lambda (o) (>= (second o) 2)) orders))))
```

(a) Say in one sentence what `revenue` computes. Then evaluate `(revenue orders)` in the substitution model, inside out, giving the value of each of the three stages.

(b) Every order of two or more items gets 10% off. Add a stage to the pipeline that applies the discount, and say in one sentence why it has to come after the `filter`.

(c) Write `revenue2`, which computes the same total as `revenue` with a single `foldl` and no `map` or `filter`. In one sentence each: what does `revenue2` save, and what does `revenue` make easier to read?

Problem 3.5 (A fold, audited). Consider applicants screened by an expensive test, of whom the first few who pass are invited. The function stops as soon as it has enough:

```
(define (first-passing n pass? applicants)
  (cond [(= n 0) '()]
        [(empty? applicants) '()]
        [(pass? (first applicants))
         (cons (first applicants)
               (first-passing (- n 1) pass? (rest applicants)))]
        [else (first-passing n pass? (rest applicants))]))
```

```
(first-passing 2 even? '(2 4 5 6 8)) ; '(2 4)
(first-passing 3 even? '(1 2 3 4))   ; '(2 4)
```

Four clauses is a lot, and someone suggests that a `foldl` would be shorter.

(a) AI allowed Ask an assistant to rewrite it with `foldl`, and record what it produces. Any follow-up you put to it belongs here too, and in the transcript. Paste the definition above, and then a prompt along these lines:

> Rewrite `first-passing` in Racket with `foldl` and no explicit recursion, to make it shorter.

Use a **basic, free** model, not the strongest you can reach.

(b) No AI Check its answer by hand on `(first-passing 2 even? '(2 4 5 6 8))`: trace it by the two equations of `foldl`, give the value, and count how many times `even?` is called. Do the same count for the original.

(c) No AI Was the rewrite an optimisation? In two or three sentences: what the original does that no `foldl` version can, and which version you would keep.

Use of AI assistants. Part (a) only. Alongside your answer file, submit `transcript.md`: the whole exchange, your prompts and the assistant's replies, verbatim and unedited. Parts (b) and (c) are answered in the answer file itself, in your own words, like every other problem on this sheet. You may **not** submit the assistant's answer as your own, and an answer to (b) with no trace shown scores nothing. If you use no assistant, do the rewrite yourself in (a) and answer (b) about your own attempt.